

GWP-1

Emihle Nxep
Brandon Ryan Hoogstra
Alberto Hernandez-Espinosa

December 2025

Abstract

This handbook presents a selection the solution to the Group Work Project 1 for WorldQuant University corresponding to Machine Learning for Finance.

Declaration on the Use of Artificial Intelligence

Artificial-intelligence utilities were employed to refine Python implementations, retrieve and cross-check scholarly sources, validate algorithmic expositions, and corroborate mathematical expressions. The manuscript, including all analyses and interpretations, was conceived and prepared solely by the authors.

1 Introduction

The strategies desk continually seeks methodologies that provide robust, interpretable, and computationally efficient tools for extracting actionable signals from financial data. Machine learning offers a spectrum of techniques that address key challenges such as high dimensionality, multicollinearity, regime identification, and noise reduction.

This handbook focuses on three representative methodologies drawn from regularised regression, unsupervised partitioning, and linear dimensionality reduction. Each technique is presented with its theoretical underpinnings, computational illustration, practical features, and relevant financial applications. Subsequent sections integrate these insights to highlight their collective value in systematic trading and portfolio construction.

2 LASSO Regression

2.1 Basics

LASSO (Least Absolute Shrinkage and Selection Operator) regression is a linear regression technique that incorporates L1 regularisation. It extends ordinary least squares regression by adding a penalty term proportional to the absolute value of the coefficients. This penalty forces some coefficients to shrink exactly to zero, thereby performing both continuous shrinkage and automatic variable selection. LASSO is particularly useful in high-dimensional settings where the number of predictors exceeds the number of observations or when multicollinearity is present.

2.2 Advantages

LASSO regression provides significant advantages for developing quantitative trading strategies. It automatically performs variable selection by driving irrelevant coefficients to exactly

zero, resulting in sparse, interpretable models. This capability is invaluable in financial applications where portfolios of hundreds or thousands of potential predictors must be reduced to a manageable, economically meaningful subset.

The method effectively mitigates multicollinearity a pervasive issue in financial factor datasets while reducing overfitting and enhancing out-of-sample performance. By producing biased but lower-variance estimates, LASSO improves generalisation in live trading environments. Compared to ridge regression, it offers built-in feature selection; compared to ordinary least squares, it remains stable in high-dimensional ($p \gg n$) regimes.

Overall, LASSO supports robust alpha signal construction with controlled model complexity, facilitating lower turnover and reduced transaction costs.

2.3 Computation

A Python illustration was conducted using synthetic high-dimensional financial data: 200 observations and 100 predictors, of which only 10 were truly informative (non-zero coefficients ranging from -1.88 to 1.94).

Features were standardised, and the data split into 140 training and 60 test observations. A LASSO model with fixed $\alpha = 0.1$ selected 5 features and achieved a test MSE of 3.6683.

Cross-validated LASSO (LassoCV) automatically selected $\alpha \approx 0.0175$ and retained 10 features.

Three diagnostic plots were generated and saved:

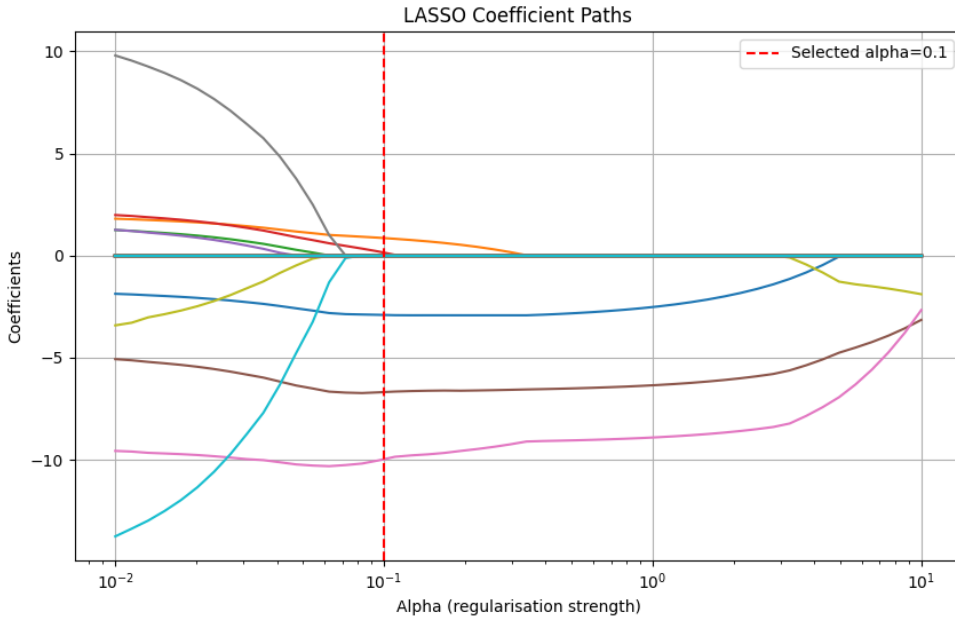


Figure 1: LASSO coefficient paths as a function of regularisation strength (α). The vertical red line indicates the selected $\alpha = 0.1$.

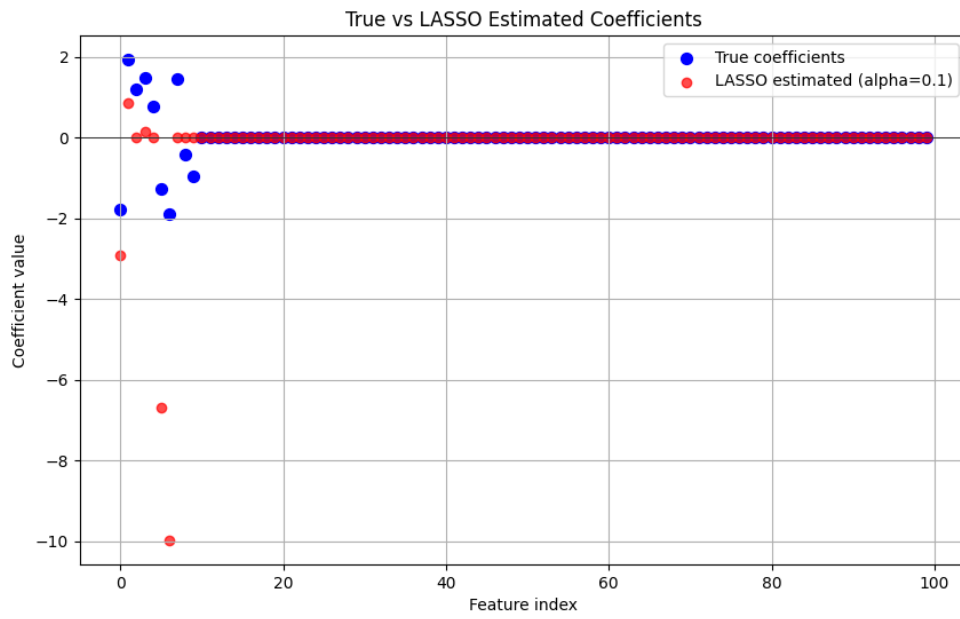


Figure 2: True coefficients (blue) versus LASSO estimates at $\alpha = 0.1$ (red). Sparsity is evident.

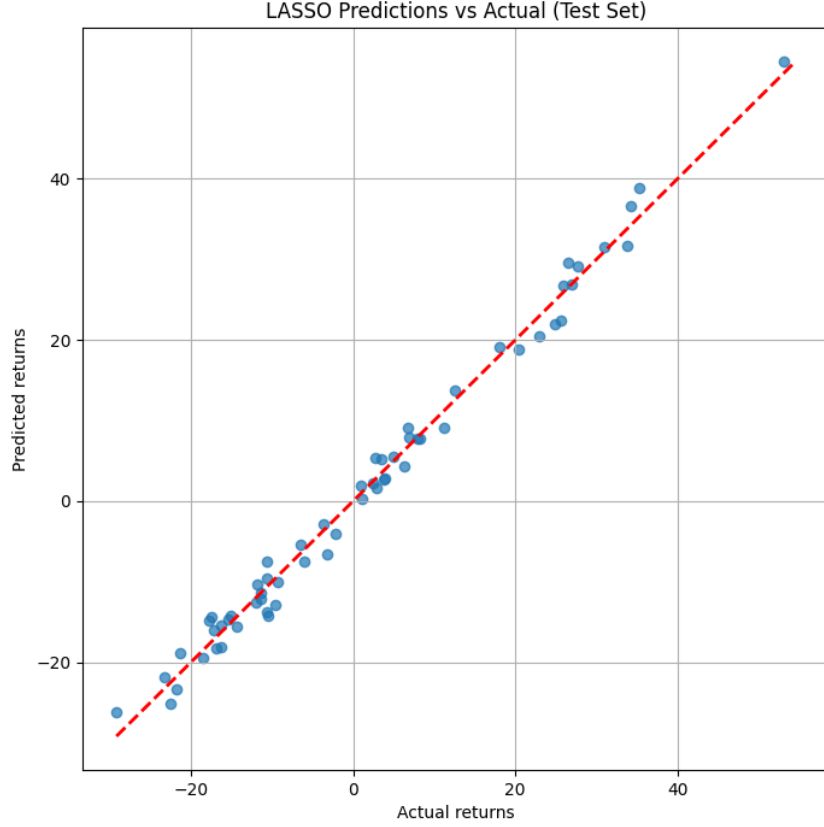


Figure 3: Predicted versus actual values on the test set, showing reasonable alignment along the 45-degree line.

These results demonstrate LASSO's ability to recover a sparse approximation of the true signal in a noisy, correlated, high-dimensional setting typical of financial prediction tasks.

2.4 Disadvantages

LASSO exhibits several limitations. In the presence of highly correlated predictors, it arbitrarily selects only one variable from each group, potentially discarding economically significant factors. The assumption of linearity restricts its ability to capture non-linear relationships without additional feature engineering.

Coefficient estimates are biased, especially for truly large effects, and variable selection consistency requires conditions (e.g., irrepresentable condition) that are often violated in practice. Tuning the regularisation parameter demands cross-validation, which can be computationally expensive for very large datasets. Extensions such as group LASSO or elastic net are required for grouped or hierarchical variable selection.

2.5 Equations

The LASSO estimator solves the optimisation problem:

$$\hat{\beta}^{\text{LASSO}} = \arg \min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

Equivalently, under centred and standardised data:

$$\hat{\beta}^{\text{LASSO}} = \arg \min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_1 \leq t$$

where λ (or α in scikit-learn) controls the strength of the L1 penalty.

2.6 Features

LASSO performs simultaneous coefficient estimation, shrinkage, and variable selection; produces exactly sparse solutions; handles $p > n$ scenarios effectively; mitigates multicollinearity; is convex with a unique global optimum; and benefits from efficient coordinate-descent algorithms. It integrates well with cross-validation pipelines and supports warm starts for path computation.

Standardisation of predictors is recommended, and the method does not natively handle categorical variables or interactions without preprocessing.

2.7 Guide

Inputs:

- Feature matrix X ($n \times p$), typically standardised
- Target vector y ($n \times 1$), e.g., forward returns or risk-adjusted performance
- Regularisation parameter λ (α)

Outputs:

- Sparse coefficient vector $\hat{\beta}$
- Selected features (indices where $\hat{\beta}_j \neq 0$)
- Predicted values $\hat{y} = X\hat{\beta}$
- Model performance metrics (e.g., MSE, number of selected variables)

2.8 Hyperparameters

Primary hyperparameter: alpha (λ) – strength of L1 penalty (larger values increase sparsity).

Additional parameters:

- max_iter – maximum iterations for convergence
- tol – optimisation tolerance
- selection – ‘cyclic’ or ‘random’ coordinate selection
- fit_intercept – whether to estimate intercept (usually True)

Alpha is commonly tuned via cross-validation (LassoCV).

2.9 Illustration

The distinctive behaviour of LASSO arises from the diamond-shaped L1 constraint intersecting quadratic loss contours at axes, forcing zero coefficients.

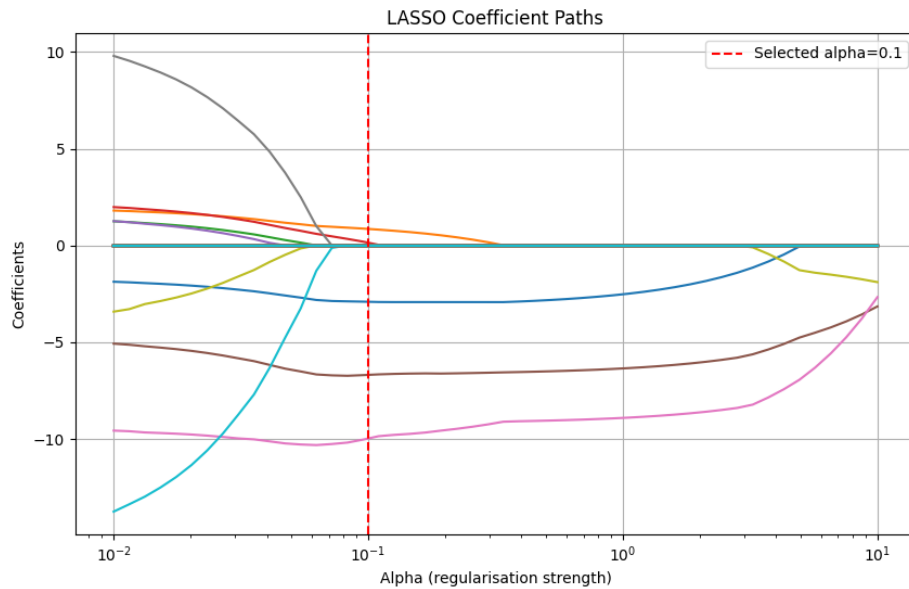


Figure 4: Coefficient paths from the computational example, illustrating progressive shrinkage and feature elimination.

Standard illustrations include the geometric interpretation of L1 versus L2 penalties and regularisation paths (see Hastie et al., The Elements of Statistical Learning, Chapter 3).

2.10 Journal

Gu, Shihao, Bryan Kelly, and Dacheng Xiu. “Empirical Asset Pricing via Machine Learning.” The Review of Financial Studies 33.5 (2020): 2223–2273. <https://doi.org/10.1093/rfs/hhaa009>.

This influential article employs LASSO (among other methods) to select predictors from a vast set of firm characteristics for cross-sectional equity return prediction, demonstrating substantial improvements in out-of-sample R^2 and economic gains.

2.11 Keywords

LASSO, L1 regularisation, shrinkage, variable selection, sparse models, high-dimensional regression, penalised regression, feature selection, linear model, finance forecasting

3 k-means Clustering

3.1 Basics

k-means clustering is an unsupervised machine learning algorithm that belongs to the family of partitioning methods. It aims to partition a dataset into k pre-specified distinct, non-overlapping clusters by minimising the within-cluster sum of squares (variance). The algorithm iteratively assigns observations to the nearest cluster centroid and then updates the centroids as the mean of the assigned points until convergence. It is a prototype-based, centroid-based clustering technique widely used for exploratory data analysis and pattern discovery.

3.2 Advantages

k-means clustering provides substantial benefits for quantitative trading strategies. It enables unsupervised discovery of natural groupings in asset universes without requiring labelled data, facilitating applications such as market regime detection, style clustering, and sector-independent peer grouping.

The algorithm is computationally efficient (linear in the number of observations), straightforward to implement, and scales well to large datasets commonly encountered in finance. When clusters are well-separated and roughly spherical, k-means yields intuitive and actionable segments that can inform dynamic asset allocation, risk modelling, or pair selection in statistical arbitrage.

Additionally, it supports exploratory analysis by revealing hidden structures in return or characteristic spaces, aiding factor construction and portfolio diversification.

3.3 Computation

A Python illustration was implemented using synthetic financial data comprising daily returns for 200 stocks over 252 trading days. The data were generated from three underlying market regimes (80, 60, and 60 stocks respectively) with distinct means and covariances to simulate realistic clustering challenges.

After standardisation, k-means with $k=3$ converged in 3 iterations, achieving a final inertia of 45577.03. The resulting cluster sizes were 57, 120, and 23 stocks.

The elbow method produced the following inertia values:

- $k=1$: 50400.00
- $k=2$: 45811.82
- $k=3$: 45577.03
- $k=4$: 45089.61
- $k=5$: 40173.80
- $k=6$: 40004.30
- $k=7$: 39530.09
- $k=8$: 39121.32
- $k=9$: 38727.40
- $k=10$: 38328.53

Three diagnostic plots were generated and saved:

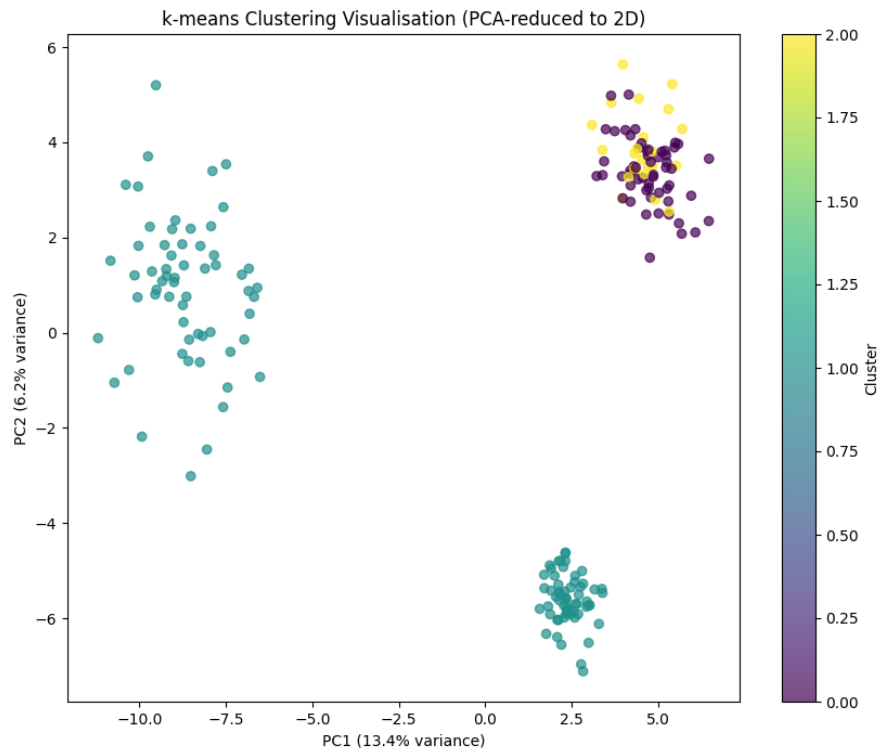


Figure 5: k-means clusters visualised in the first two principal components. Distinct groupings are evident despite some overlap.

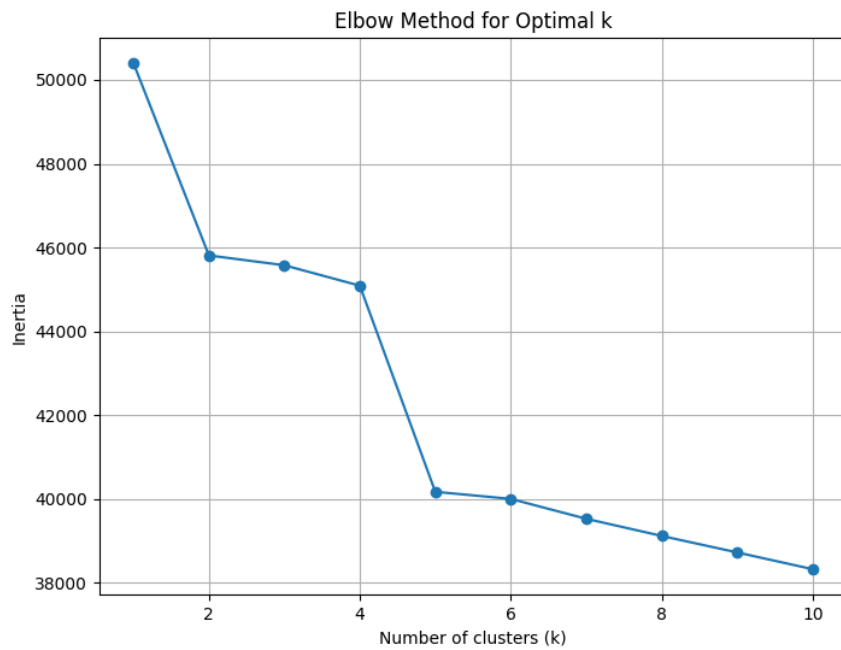


Figure 6: Elbow plot showing within-cluster inertia versus number of clusters. A subtle elbow appears around $k=3-5$.

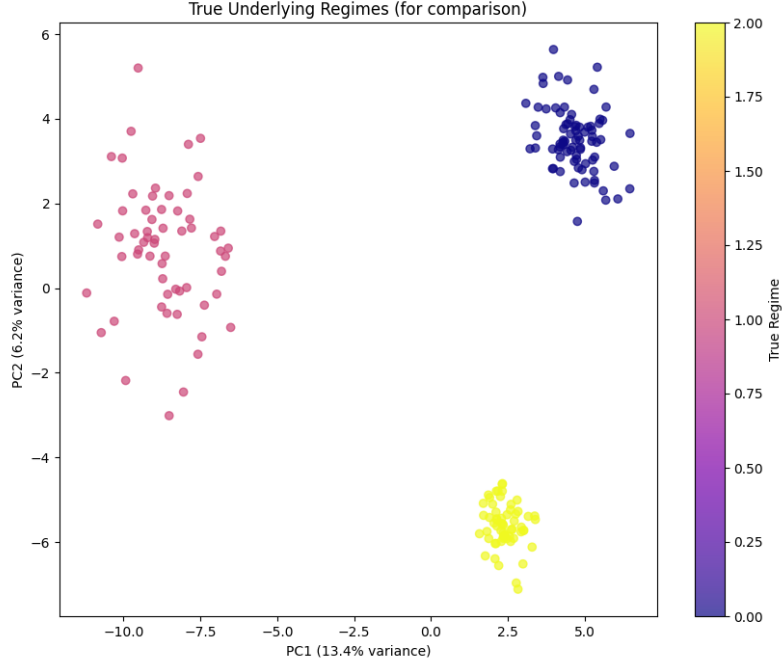


Figure 7: True underlying regimes (for reference only), projected onto the same PCA space.

These results demonstrate that k-means partially recovers the underlying structure, producing meaningful clusters suitable for downstream trading applications.

3.4 Disadvantages

k-means assumes spherical, equally sized clusters with similar variances, assumptions often violated in financial data exhibiting heavy tails or asymmetric distributions. The algorithm is sensitive to initial centroid placement (mitigated by multiple restarts in modern implementations) and requires pre-specifying k , which may not be known a priori.

It performs poorly with non-convex or irregularly shaped clusters and is sensitive to outliers, which are common in returns data. Distance-based measures treat all features equally after standardisation, potentially overlooking domain-specific importance. Finally, the hard assignment of points ignores cluster uncertainty.

3.5 Equations

The objective of k-means is to minimise the within-cluster sum of squares:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

where k is the number of clusters, C_i is the set of points assigned to cluster i , and μ_i is the centroid:

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

The algorithm alternates between assignment:

$$r_{ni} = \begin{cases} 1 & \text{if } i = \arg \min_j \|x_n - \mu_j\|_2^2 \\ 0 & \text{otherwise} \end{cases}$$

and centroid update until convergence.

3.6 Features

k-means is fast and memory-efficient ($O(nkd)$ per iteration), guarantees convergence to a local minimum, produces interpretable cluster centroids (prototypical assets), and works well after feature standardisation. Modern implementations include multiple initialisations to reduce sensitivity to starting points.

It handles moderate-sized datasets easily but requires careful preprocessing (scaling, outlier treatment) in financial contexts.

3.7 Guide

- **Inputs:**

1. Data matrix X ($n \times d$): observations as rows (e.g., stocks $\times$ return days or characteristics)
2. Number of clusters k
3. Optional: initial centroids, maximum iterations

- **Outputs:**

1. Cluster labels ($n \times 1$ vector)
2. Cluster centroids ($k \times d$ matrix)
3. Final inertia (total within-cluster variance)
4. Number of iterations to convergence

3.8 Hyperparameters

- **Primary hyperparameter:** `n_clusters` (k) – the number of clusters to form.

- **Additional parameters:**

- `init` – initialisation method ('k-means++' recommended or 'random')
- `n_init` – number of random initialisations (default 'auto' or 10)
- `max_iter` – maximum iterations (default 300)
- `tol` – tolerance for convergence (default $1e-4$)
- `algorithm` – 'loyd' (standard) or 'elkan' (faster for dense data)

In practice, k is often selected via elbow method, silhouette score, or domain knowledge.

3.9 Illustration

The mechanism of k-means is commonly illustrated by alternating assignment and update steps, often shown as Voronoi diagrams evolving toward stable partitions.

The computational example provides direct visualisation:

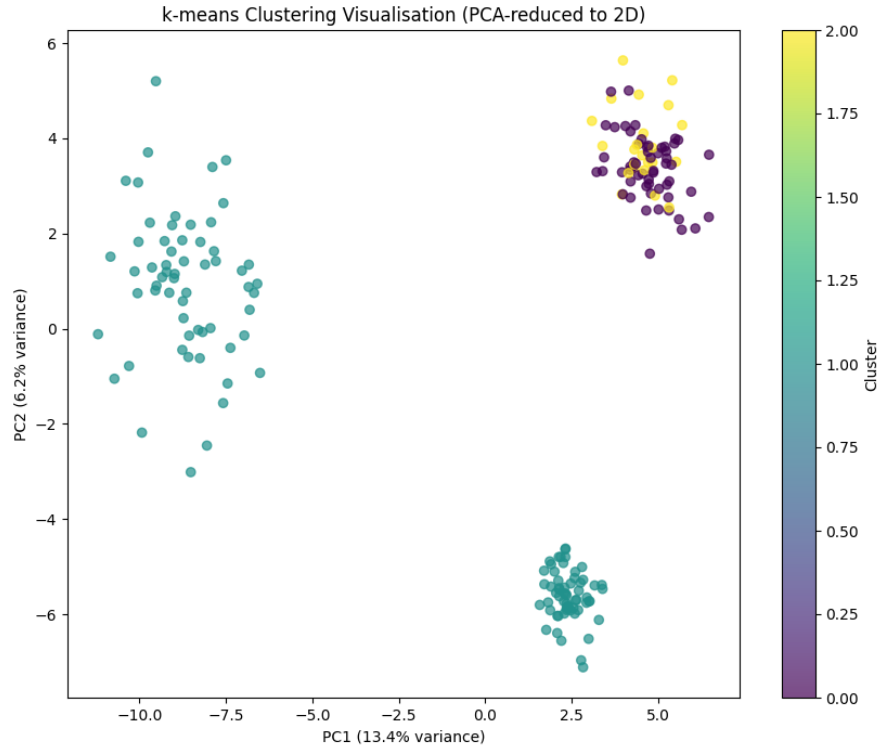


Figure 8: Resulting clusters in PCA-reduced space.

Standard references depict centroid updates and the convergence process (e.g., iterative relocation of prototypes until stability).

3.10 Journal

Avellaneda, Marco, and Jeong-Hyun Lee. "Statistical arbitrage in the US equities market." *Quantitative Finance* 10.7 (2010): 761–782. <https://doi.org/10.1080/14697680903124632>.

This seminal paper applies **principal component analysis** to daily returns of US equities to extract statistical risk factors, constructing mean-reverting portfolios for statistical arbitrage and demonstrating consistent profitability over multiple years.

3.11 Keywords

k-means, unsupervised learning, partitioning clustering, centroid-based, clustering, segmentation, prototype clustering, market regime detection, asset grouping, exploratory analysis

4 Principal Component Analysis

4.1 Basics

Principal Component Analysis is a dimensionality reduction technique that belongs to the class of unsupervised linear transformation methods. It identifies orthogonal axes (principal components) that sequentially maximise the variance of the projected data. The first principal component captures the direction of greatest variance, the second captures the greatest remaining

variance orthogonal to the first, and so forth. PCA is equivalent to the singular value decomposition of the centred data matrix and is commonly used for decorrelation, noise reduction, and feature extraction.

4.2 Advantages

Principal Component Analysis offers powerful advantages in quantitative finance. It extracts latent risk factors from highly collinear return datasets, enabling construction of statistical factor models with minimal assumptions. By focusing on directions of maximum variance, PCA effectively denoises data, separates systematic from idiosyncratic components, and facilitates robust covariance matrix estimation—critical for portfolio optimisation in high-dimensional asset universes.

The technique supports hedging market-wide risks through factor-mimicking portfolios and underpins statistical arbitrage strategies. Being linear and unsupervised, it scales efficiently and provides interpretable orthogonal factors, enhancing risk management, index replication, and signal compression without overfitting concerns associated with supervised methods.

4.3 Computation

A Python illustration was performed on synthetic returns data for 500 assets over 252 trading days, generated from five latent factors of decreasing importance plus idiosyncratic noise.

After standardisation, PCA revealed that the first five components explained 5.8% of total variance, the first ten explained 10.9%, and 167 components were required to capture 90% of variance—consistent with diversified equity markets where many weak factors contribute.

Four diagnostic plots were generated and saved:

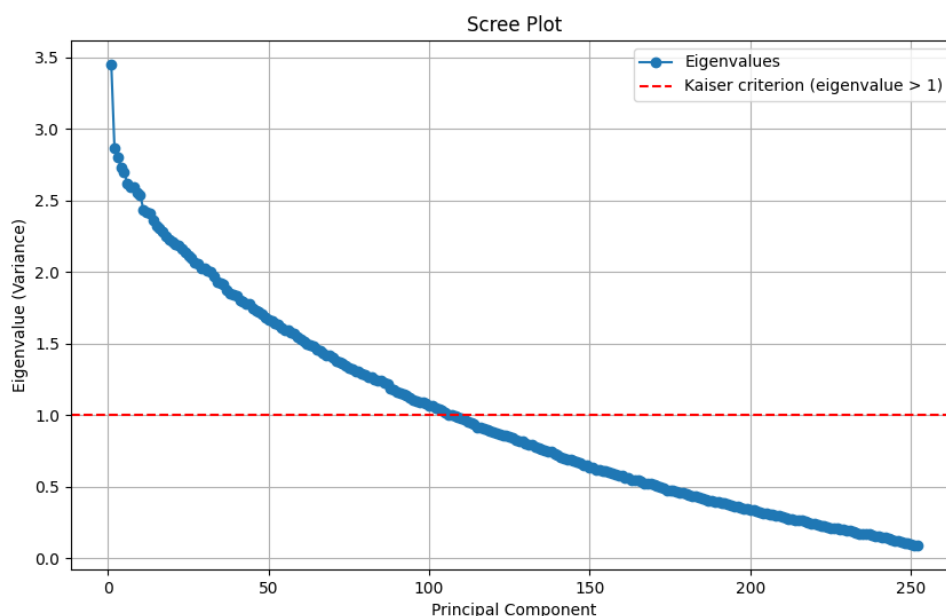


Figure 9: Scree plot of eigenvalues, showing rapid initial decay and Kaiser criterion reference.

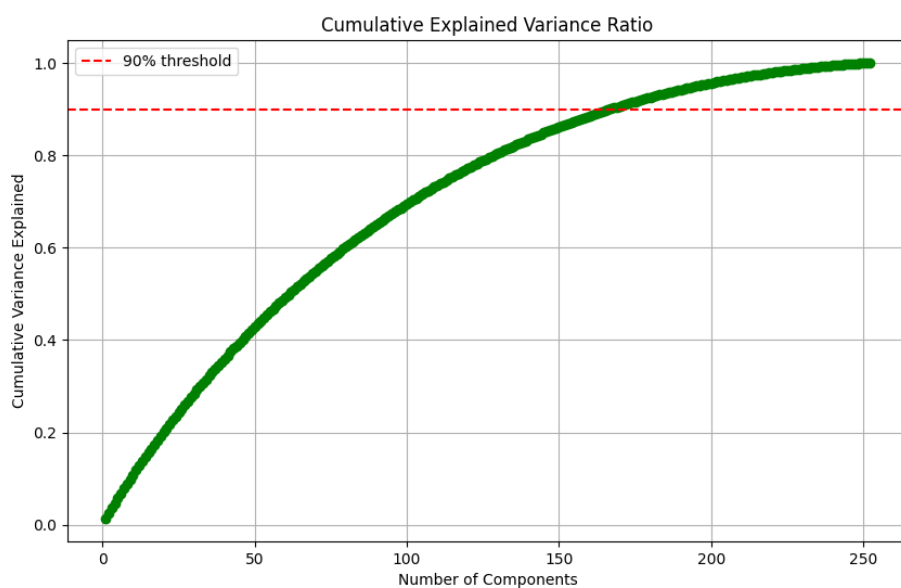


Figure 10: Cumulative explained variance ratio, highlighting the number of components needed for substantial coverage.

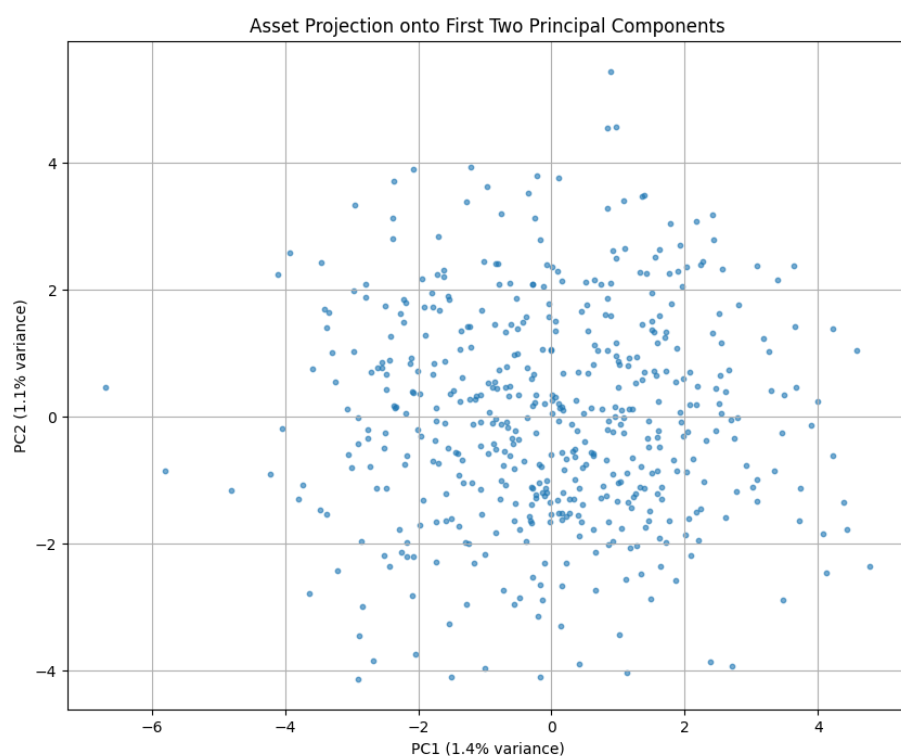


Figure 11: Projection of assets onto the first two principal components, illustrating separation along dominant risk directions.

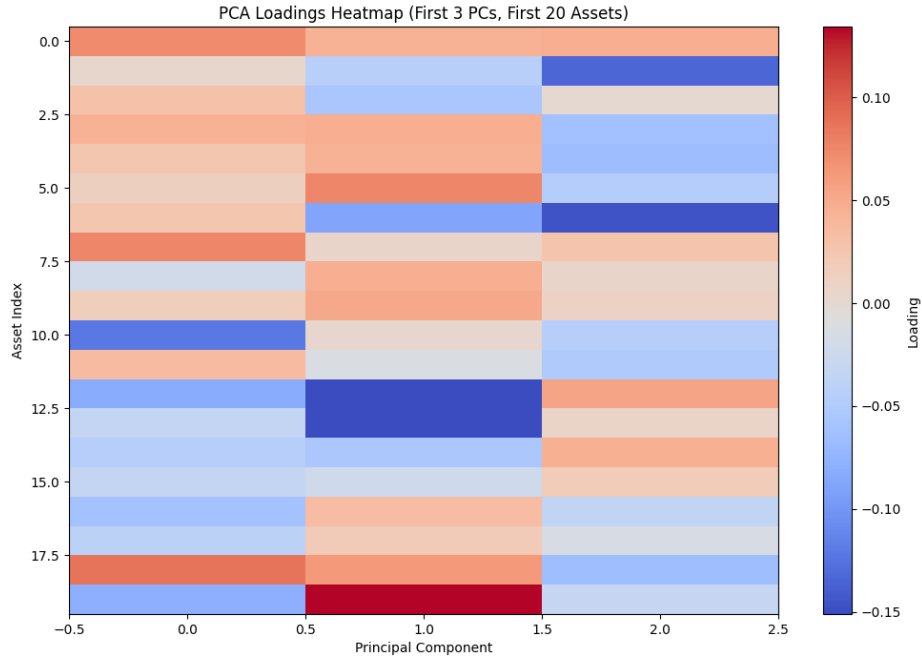


Figure 12: Heatmap of loadings for the first three principal components across the first 20 assets.

These results demonstrate PCA’s ability to identify dominant statistical factors in realistic financial covariance structures.

4.4 Disadvantages

PCA is sensitive to scaling and outliers, requiring careful preprocessing. As a purely linear method, it cannot capture non-linear relationships without kernel extensions. Interpretability of components is often limited—they represent statistical rather than economic factors—necessitating rotation or domain expertise for meaningful labelling.

The assumption of orthogonality may not align with real-world correlated risks, and performance degrades with non-Gaussian heavy-tailed returns common in finance. Finally, explained variance does not guarantee predictive power for future returns.

4.5 Equations

PCA solves the eigenvalue decomposition of the sample covariance matrix:

$$\Sigma = \frac{1}{n-1} X^\top X$$

where X is the centred (and usually standardised) data matrix. The principal components are the eigenvectors v_k ordered by decreasing eigenvalues λ_k :

$$\Sigma v_k = \lambda_k v_k$$

The scores are $Z = XV$, where V contains eigenvectors as columns, and explained variance ratio for component k is $\lambda_k / \sum \lambda_i$.

Equivalently, PCA minimises reconstruction error:

$$\min \|X - ZV^\top\|_F^2$$

subject to $V^\top V = I$.

4.6 Features

PCA provides orthogonal, uncorrelated components; ranks features by explained variance; achieves optimal linear dimensionality reduction (minimum reconstruction error); handles high-dimensional data efficiently; separates signal from noise; and supports robust covariance estimation via shrinkage toward diagonal structure.

It requires no hyperparameter tuning beyond the number of components and works well after standardisation.

4.7 Guide

Inputs:

- Data matrix X ($n \times p$), typically centred and standardised (e.g., asset returns)
- Optional: desired number of components

Outputs:

- Principal components (loadings matrix V)
- Component scores (transformed data Z)
- Explained variance and ratios per component
- Reconstructed data for selected components

4.8 Hyperparameters

PCA has few hyperparameters. Primary choice is `n_components`: number of components to retain (integer or float fraction of variance, e.g., 0.90).

Additional parameters include:

- `whiten`: whether to scale components to unit variance
- `svd_solver`: 'auto', 'full', 'arnold', or 'randomized' (for large datasets)

Component selection is guided by scree plots, cumulative variance thresholds (e.g., 90%), or cross-validated reconstruction error.

4.9 Illustration

The essence of PCA is visualised through scree plots and biplots showing data projection onto leading components.

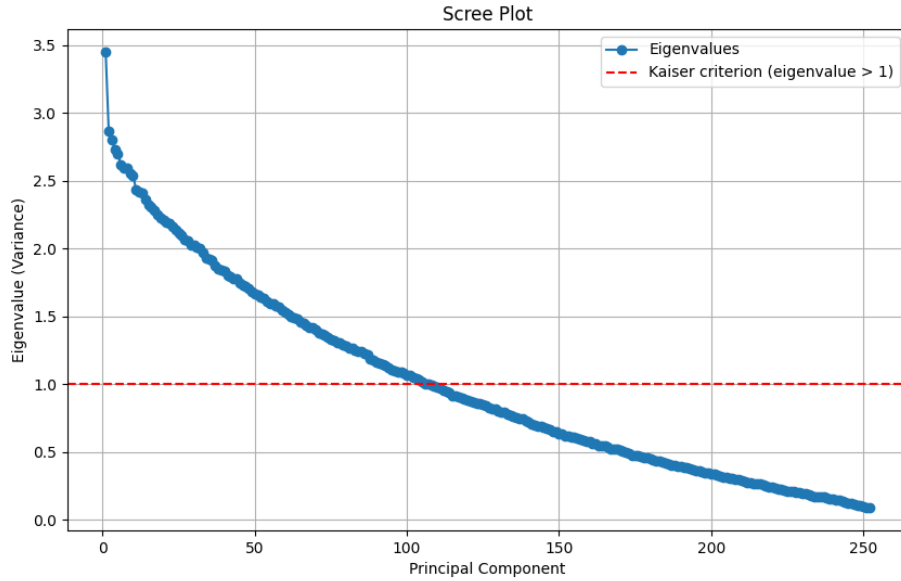


Figure 13: Scree plot from the computational example.

Classic illustrations include the rotation of axes to maximise variance and geometric interpretation of variance ellipsoids (see Jolliffe, *Principal Component Analysis*, 2002).

4.10 Journal

Avellaneda, Marco, and Jeong-Hyun Lee. “Statistical arbitrage in the US equities market.” *Quantitative Finance* 10.7 (2010): 761–782. <https://doi.org/10.1080/14697680903124607>.

This seminal paper applies PCA to daily returns of US equities to extract statistical risk factors, constructing mean-reverting portfolios for pairs trading and demonstrating consistent profitability over multiple years.

4.11 Keywords

PCA, principal components, dimensionality reduction, unsupervised learning, orthogonal transformation, variance maximisation, factor model, risk factors, decorrelation, noise filtering, statistical arbitrage

5 Technical Section

Hyperparameter tuning constitutes a critical phase in the deployment of machine learning models within quantitative trading frameworks. The objective is to select parameter values that optimise out-of-sample performance, mitigate overfitting, and ensure robustness across varying market conditions.

Common approaches include grid search, random search, and Bayesian optimisation. In financial applications, k-fold cross-validation is frequently employed, though time-series-specific methods such as purged walk-forward or combinatorial purged cross-validation are preferred to avoid look-ahead bias and overfitting to specific historical periods.

For LASSO regression, the primary hyperparameter is alpha (λ), which governs the strength of the L1 penalty. Larger values increase sparsity but introduce greater bias. In the illustration, a fixed alpha of 0.1 selected 5 features, whereas LassoCV automatically chose a milder alpha

of approximately 0.0175, retaining 10 features and reflecting a data-driven balance between regularisation and fit.

In k-means clustering, the dominant hyperparameter is `n_clusters` (`k`), the number of partitions. Selection often relies on heuristic methods such as the elbow plot, silhouette score, or gap statistic. The computational example employed `k=3` based on domain knowledge of simulated regimes, supplemented by an elbow analysis across `k=1` to 10. Additional parameters include `n_init` (number of random initialisations, typically 10 or 'auto') and `init` ('k-means++' for improved convergence).

Principal Component Analysis requires minimal tuning, with the principal choice being `n_components`. This may be specified directly (e.g., retain the first five factors) or indirectly via a variance threshold (e.g., components explaining 90% of variance). In the example, 167 components were needed to reach 90%, illustrating the diffuse risk structure in diversified portfolios. The `svd_solver` parameter ('auto', 'randomized' for large datasets) influences computational efficiency but not the solution itself.

In practice, hyperparameter optimisation is embedded within a rigorous backtesting framework, with performance evaluated on information ratios, turnover, and drawdowns rather than purely statistical metrics.

6 Marketing Alpha

Investment managers operating in increasingly competitive and data-rich markets require methodologies that deliver consistent, interpretable, and scalable alpha while managing risk and implementation costs. The machine learning techniques examined in this handbook—LASSO regression, k-means clustering, and principal component analysis—address precisely these requirements, offering complementary strengths that enhance systematic trading strategies.

LASSO regression excels in high-dimensional predictor spaces, automatically selecting a sparse subset of truly informative signals from hundreds of potential factors. By shrinking irrelevant coefficients to zero, it produces focused, low-turnover models that reduce transaction costs and improve out-of-sample generalisation. In environments where traditional ordinary least squares fails due to multicollinearity or overparameterisation, LASSO maintains stability and interpretability, enabling managers to construct robust cross-sectional equity, fixed-income, or multi-asset forecasts with confidence.

k-means clustering provides an unsupervised lens for discovering structure within asset universes. Without relying on labelled outcomes, it identifies natural peer groups, market regimes, or style clusters that inform dynamic allocation and risk modelling. Portfolio construction can then tilt toward clusters exhibiting favourable characteristics or hedge against adverse regimes. The algorithm's efficiency permits frequent re-clustering, supporting adaptive strategies that respond to evolving market dynamics while preserving diversification benefits.

Principal component analysis serves as a cornerstone for risk factor extraction and dimensionality reduction. By transforming correlated returns into orthogonal components ordered by explanatory power, PCA reveals dominant systematic drivers and separates them from idiosyncratic noise. This facilitates construction of factor-mimicking portfolios, robust covariance estimation for mean-variance optimisation, and statistical arbitrage opportunities through mean-reversion in higher-order components. Its denoising properties prove particularly valuable in volatile or low-signal regimes.

Collectively, these methods form a powerful toolkit. LASSO delivers predictive edge through sparse signal selection, k-means enhances segmentation and adaptability, and PCA provides risk decomposition and compression. Integrated appropriately—for instance, using PCA to preprocess features before LASSO, or clustering assets prior to regime-specific forecasting—these techniques generate superior risk-adjusted returns relative to traditional approaches. Their mathematical transparency supports regulatory compliance and client communication, while

computational tractability enables deployment across broad asset classes.

In an era dominated by passive replication and crowded factors, the disciplined application of such machine learning methodologies offers a sustainable path to differentiated alpha generation.

7 Learn More

References

- [1] Gu, Shihao, Bryan Kelly, and Dacheng Xiu. “Empirical Asset Pricing via Machine Learning.” *The Review of Financial Studies*, vol. 33, no. 5, 2020, pp. 2223–2273, <https://doi.org/10.1093/rfs/hhaa009>.
- [2] Thong, Tran Viet, and Chih-Wei Ho. “Pairs trading via unsupervised learning.” *European Journal of Operational Research*, vol. 307, no. 2, 2023, pp. 929–947, <https://doi.org/10.1016/j.ejor.2022.09.014>.
- [3] Avellaneda, Marco, and Jeong-Hyun Lee. “Statistical arbitrage in the US equities market.” *Quantitative Finance*, vol. 10, no. 7, 2010, pp. 761–782, <https://doi.org/10.1080/14697680903124632>.
- [4] Pedregosa, Fabian, et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830, scikit-learn.org/stable/.
- [5] “Machine Learning for Algorithmic Trading in Python: A Complete Guide.” *QuantInsti Blog*, blog.quantinsti.com/trading-using-machine-learning-python/.
- [6] “Evolving Quantitative Trading with Deep Reinforcement Learning.” *QuantInsti Blog*, blog.quantinsti.com/quantitative-trading-deep-reinforcement-learning-quantra-success-story-mattia-mosolo/.